Indie Devs Documentation

Alejandro Garcia Jr
Garrett DeYong
Nicholas Nolen
William Waweru
Gan-Orgil Gantumur

October 14, 2026

Contents

Change Log **2**

1 Introduction **3**

2 Project Overview **4**
2.1 Product Perspective 4
2.2 Assumptions and Dependencies 4

3 Requirements Analysis **5**

4 User Story **6**

5 Key Technical Challenges **8**

6 What's New? **9**

7 Use Cases **10**
7.1 Alejandro Garcia Jr – Use Cases 10
7.1.1 UC: Search for Artist – Fan 10
7.1.2 UC: Review Report – Admin 11
7.1.3 UC: Create Event – Artist 12
7.2 Garrett DeYong – Use Cases 13
7.2.1 UC: Make a Post – Artist 13
7.2.2 UC: Message Another User (Beginning a New Conversation) 14
7.2.3 UC: Monitor Comments – Artist 15
7.3 Nicholas Nolen – Use Cases 16
7.3.1 UC: Update Event 16
7.3.2 UC: Unban Fan 17
7.3.3 UC: Recieve Reccomendation 17
7.4 Gan-Orgil Gantumur – Use Cases 18
7.4.1 UC: Suspending Artist Account – Admin 18
7.4.2 UC: Comment on Artwork – Fan 19
7.4.3 UC: Approve Artist Verification Application – Admin 20
7.5 William Waweru – Use Cases 20
7.5.1 UC: Follow Artist – Fan 20
7.5.2 UC: Manage Fan Profile – Fan 21
7.5.3 UC: Report Inappropriate Content – Fan 22

8 Sequence Diagrams **24**
8.1 Garrett DeYong – Sequence Diagrams . . . 24
8.2 Nicholas Nolen – Sequence Diagrams . . . 26
8.3 Gan-Orgil Gantumur – Sequence Diagrams . . . 29
8.4 Alex Garcia – Sequence Diagrams . . . 30
8.5 William Waweru – Sequence Diagrams . . . 32
8.5.1 Follow Artist – Fan . . . 32
8.5.2 Manage Fan Profile – Fan . . . 32
8.5.3 Report Inappropriate Content – Fan . . . 33

9 Use Case Diagram **34**

10 Domain Model **35**

11 Operation Contracts **36**

12 Test Cases **38**
12.1 Test Cases for Iteration 1 . . . 38
12.2 Test Cases for Iteration 2 . . . 38

13 UI Mockups **39**
13.1 Make a Post – Artist . . . 39
13.2 Search for Artist – Fan . . . 41
13.3 Create Event – Artist . . . 42

1. Introduction

ArtistConnect is a web-based platform designed to help independent artists share their creative work and build meaningful relationships with their audiences. Artists often rely on several separate services to publish media, announce events, and communicate with supporters. Fans, meanwhile, may find it difficult to discover emerging artists in their own communities.

ArtistConnect brings these activities together in one place. Artists can maintain profiles, publish multimedia posts, create events, and interact with fans. Fans can search for artists by name, style, or location, follow creators, and engage with their content. Administrators support a safe community by reviewing reports and managing accounts. The project aims to make independent creative communities easier to discover, participate in, and sustain.

2. Project Overview

ArtistConnect provides a shared online environment for independent artists, fans, and platform administrators. Its core capabilities include artist discovery, multimedia publishing, event promotion, community interaction, direct messaging, and content moderation. Rather than focusing on a single medium, the platform accommodates different kinds of creative work and emphasizes connections between artists and their local audiences.

2.1 Product Perspective

ArtistConnect is intended to operate as a standalone web application accessible through a browser. The user interface provides distinct workflows for artists, fans, and administrators. The envisioned system uses an application layer to handle business rules and a database to store user accounts, artist profiles, posts, events, and relationships between users. The project also anticipates deployment through a cloud hosting service.

The current user-interface previews demonstrate representative workflows, including creating a post, searching for artists, and creating an event. These previews illustrate the intended user experience; browser-local demonstration data should not be mistaken for completed backend persistence, live notifications, or deployed production functionality.

2.2 Assumptions and Dependencies

The project assumes that users can access a modern web browser and that registered accounts will be associated with appropriate roles and permissions. Artist discovery depends on artists providing searchable profile information, such as their name, creative style, and location. Event creation depends on artists supplying sufficient event details, and publication of ticket pricing may require administrative approval.

The full system depends on the availability of backend services, persistent data storage, and hosting infrastructure. Multimedia uploads will also require suitable file validation and storage. Features involving live messaging, notifications, recommendations, or automated moderation may depend on additional services and must be distinguished from functionality demonstrated only in the interface prototype.

3. Requirements Analysis

Customer / Stakeholder Perspective: Small independent artists need a platform to host their art and foster deeper connections with their fan communities. As an artist, I want a hybrid social media and art hosting site to upload my music, images, and stories, promote events, and interact directly with my supporters through messages and videos. As a fan, I want to discover new artists near me, follow their updates, attend their shows, and interact with other members of the community. As an administrator, I need tools to moderate the platform, review applications, and keep the community safe.

Functional Requirements:

- The system must support role-based access control with distinct permissions and workflows for Fans, Artists, and Administrators.
- The system must allow Artists to maintain a profile, upload multimedia content (music, art images, stories, videos), and promote their events.
- The system must enable Fans to maintain an "about" profile and discover new artists using a proximity-based radius search.
- The system must provide a messaging infrastructure allowing fans to message artists and each other.
- The system must allow Administrators to review prospective artist applications, respond to user reports, and manage/ban accounts violating the Terms of Service (ToS).

Non-Functional Requirements / Environment:

- The system must be a web-based application utilizing a 3-tier architecture (frontend web server, backend application layer, and database) to handle persistence and advanced business logic.
- The system must be deployed to a cloud hosting service such as Google Cloud Platform (GCP), Amazon Web Services (AWS), or Microsoft Azure.
- The development lifecycle must incorporate a CI/CD pipeline with mandatory pull requests.
- Any optional AI functionalities (e.g., content moderation, recommendation engines, or AI-generated art detectors) must be designed to respect user privacy, include human oversight, and feature fallback behavior.

4. User Story

- **[US-01]** As a **Fan**, I want to **search for artists using a proximity-based radius** so that **I can discover and attend local shows**. *(Maps to UC: Search for Artist)*
- **[US-02]** As a **Fan**, I want to **receive artist recommendations** so that **I can easily find new creators similar to the ones I already enjoy**. *(Maps to UC: Receive Recommendation)*
- **[US-03]** As an **Artist**, I want to **create posts with multimedia content (music, images, videos)** so that **I can share my art and stories with my fan community**. *(Maps to UC: Make a Post)*
- **[US-04]** As a **Fan**, I want to **comment on an artist's uploaded artwork or posts** so that **I can show my support and engage with the community**. *(Maps to UC: Comment on Artwork)*
- **[US-05]** As an **Artist**, I want to **monitor and manage the comments on my posts** so that **I can hide inappropriate content and report abusive users to admins**. *(Maps to UC: Monitor Comments)*
- **[US-06]** As a **User (Fan or Artist)**, I want to **send direct messages to other users** so that **I can privately communicate, collaborate, or socialize**. *(Maps to UC: Message Another User)*
- **[US-07]** As an **Artist**, I want to **create and promote upcoming events** so that **my fans know when and where to see my shows**. *(Maps to UC: Create Event)*
- **[US-08]** As an **Artist**, I want to **edit and update my existing event information** so that **my fans have the most accurate and current details if plans change**. *(Maps to UC: Update Event)*
- **[US-09]** As an **Administrator**, I want to **approve prospective artist verification applications** so that **I can ensure only legitimate, non-fake artists are granted the Artist role**. *(Maps to UC: Approve Artist Verification Application)*
- **[US-10]** As an **Administrator**, I want to **review conduct reports submitted by users** so that **I can make fair moderation decisions regarding inappropriate content**. *(Maps to UC: Review Report)*
- **[US-11]** As an **Administrator**, I want to **suspend or ban user accounts that violate the terms of service** so that **the platform remains a safe environment for artists and fans**. *(Maps to UC: Suspending Artist Account)*

- **[US-12]** As an **Administrator**, I want to **review and process ban appeals from fans** so that **I can reverse unfair or incorrect bans made by the system or other admins**. *(Maps to UC: Unban Fan)*

5. Key Technical Challenges

- **AI Feature Integration:** Integrating machine learning for moderation and recommendations poses performance and privacy challenges.
  Approach: Decouple AI processing from core traffic using scalable REST APIs and implement heuristic fallbacks to ensure uptime if models fail.
- **3-Tier Cloud Deployment:** Managing a frontend, backend, database, and strict CI/CD pipeline requires significant DevOps overhead.
  Approach: Establish automated testing and mandatory pull requests early, utilizing managed cloud services (GCP, AWS, or Azure) to minimize infrastructure management.

Key Risks

- **Scope Creep:** The extensive feature set (hybrid media, messaging, AI) risks exceeding the semester timeframe.
  Mitigation: Strictly prioritize core workflows (authentication, media, messaging) over optional AI features using agile sprints.
- **Content Moderation:** User-generated media and messaging introduce risks of Terms of Service violations.
  Mitigation: Implement robust user-reporting systems, Admin review workflows, and automated moderation filters.

Competition

- **SoundCloud:** *What it does well:* Excellent audio hosting, discovery, and timestamped comments.
  Where it falls short: Lacks visual art hosting, video interactions, and proximity-based local discovery.
- **Instagram:** *What it does well:* Massive reach for visual art, short-form videos, and messaging.
  Where it falls short: Not tailored to independent artists, lacks high-quality audio hosting, and prioritizes viral content over local community connections.

6. What's New?

Existing platforms generally focus on a single medium (e.g., SoundCloud) or cater to broad audiences (e.g., Instagram). Our project introduces a **hybrid platform tailored explicitly for small, independent, multi-disciplinary artists**.

Key Differentiators:

- **Multi-Disciplinary Hub:** Consolidates high-quality audio, visual art, and video hosting into a single profile.
- **Proximity-Based Discovery:** Replaces viral-chasing algorithms with a location-based search to foster real-world, local community connections.
- **Authenticity & Safety Focus:** Proposes specialized AI tools—like AI-generated art detectors and privacy-respecting moderation—to maintain a secure, human-centric ecosystem dedicated to independent art.

7. Use Cases

Actors Overview

- **Fan** – discovers and follows artists, messages them, and books events.
- **Artist** – manages a public profile, proposes events, and communicates with fans.
- **Admin** – moderates content and accounts by reviewing user-filed reports.

7.1 Alejandro Garcia Jr – Use Cases

7.1.1 UC: Search for Artist – Fan

Primary Actor
Fan

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests
- **Fan:** wants to quickly find artists matching their interests.
- **Artist:** wants to be discoverable by relevant fans.

Preconditions
- The Fan is authenticated and logged into the system.

Postconditions (Success Guarantee)
- A list of artists matching the Fan's search criteria was displayed.
- If the Fan selected a result, the corresponding action (profile view, chat, follow, or booking request) was recorded by the system.

Main Success Scenario
1. The Fan selects the search feature.
2. The system prompts the Fan for search criteria (name, genre/style, location).
3. The Fan enters one or more criteria and submits the search.
4. The system queries registered artists matching the given criteria.
5. The system displays a list of matching artists, each showing name, genre, location, and profile thumbnail.
6. The Fan selects an artist from the results list.
7. The system displays the selected artist's full profile.
8. The Fan chooses a follow-up action: start a direct chat, follow the artist, or book the artist.

9. The system performs the selected action and confirms it to the Fan.

Extensions 2a. The Fan submits the search with no criteria entered: the system prompts the Fan to enter at least one search criterion.

4a. No artists match the given criteria: the system displays a "no results found" message and suggests broadening the search.

6a. The Fan exits without selecting a result: the use case ends with no further action taken.

8a. The Fan chooses "Start Chat" and a conversation already exists with that artist: the system opens the existing conversation instead of creating a new one.

8b. The Fan chooses "Book": the system starts the booking-request flow (a separate use case) and, on submission, notifies the Artist that a booking request is pending.

Special Requirements
- Search results should be returned in under 2 seconds for typical query loads.
- Result lists longer than 20 artists are paginated.

Frequency of Occurrence
High – expected to be one of the most common Fan actions.

7.1.2 UC: Review Report – Admin

Primary Actor
Admin

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests
- **Admin:** needs enough context to make a fair, auditable moderation decision.
- **Reporting user (Fan or Artist):** wants inappropriate content or behavior addressed.
- **Reported user (Fan or Artist):** has an interest in fair treatment and being informed of any action taken.

Preconditions
- The Admin is authenticated with administrative privileges.
- At least one report exists with status "Pending," filed by a Fan or an Artist against a message, profile, or event.

Postconditions (Success Guarantee)
- The report's status was updated to "Resolved," with the decision and acting Admin recorded.
- Any moderation action taken (dismiss, warn, delete content, suspend/ban account) was applied to the reported content or account.
- Both the reporting user and the reported user were notified of the outcome.

Main Success Scenario 1. The Admin opens the reports queue.

2. The system displays pending reports, each showing the reporter, reported content or user, reason, and timestamp.

3. The Admin selects a report to review.
4. The system displays full report details, including the reported message, profile, or event, and the reporter's stated reason.
5. The Admin evaluates the report and selects an action: dismiss, warn the user, delete the content, or suspend/ban the account.
6. The system applies the selected action.
7. The system records the resolution (Admin identity, action taken, timestamp) and sets the report status to "Resolved."
8. The system notifies the reporting user that their report was reviewed, and notifies the reported user of any action taken against them.

Extensions 3a. The selected report was already resolved by another Admin: the system informs the Admin and returns to the queue.

5a. The Admin dismisses the report as unfounded: the system records the dismissal and notifies only the reporting user.

5b. The Admin suspends or bans the reported user's account: the system also terminates any active sessions for that account.

6a. The reported content was already removed (e.g., deleted by its owner): the system notes this and proceeds directly to logging the resolution.

Special Requirements
- The reports queue must support filtering by report type (message, profile, or event).
- All resolutions must be permanently logged for audit purposes.

Frequency of Occurrence
Low to moderate – driven by user-filed reports rather than a routine action.

7.1.3 UC: Create Event – Artist

Primary Actor
Artist

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests
- **Artist:** wants to promote and sell tickets to an upcoming event.
- **Admin:** needs to approve proposed pricing before the event is publicly visible.
- **Fan:** wants accurate, trustworthy event listings.

Preconditions
- The Artist is authenticated and has an account in good standing.

Postconditions (Success Guarantee)
- A new event was created with status "Pending Approval" and routed to the Admin queue.
- Once approved, the event's status was updated to "Published" and it became visible/searchable to Fans, who were notified if they follow the Artist.

Main Success Scenario 1. The Artist selects "Create Event."

2. The system displays the event creation form.
3. The Artist enters the event details: title, date/time, location (physical or virtual link), description, capacity, and one or more proposed ticket prices/tiers.
4. The Artist submits the event.
5. The system validates the submitted data.
6. The system creates the event with status "Pending Approval" and adds it to the Admin approval queue.
7. The system confirms to the Artist that the event was submitted for review.
8. The Admin reviews and approves the proposed pricing (see UC: Review Event Pricing).
9. The system updates the event's status to "Published," notifies the Artist of the approval, and makes the event visible/searchable to Fans, notifying the Artist's followers.

Extensions 3a. The Artist saves the event as a draft instead of submitting: the system stores it with status "Draft" and does not route it for approval.

5a. The submitted data is missing required fields or contains an invalid date/price: the system flags the offending fields and asks the Artist to correct them before resubmitting.

8a. The Admin rejects the proposed pricing: the system notifies the Artist with the rejection reason(s); the Artist may revise and resubmit, returning to step 3.

9a. The Artist edits event details after publication: the system flags the event for Admin re-approval if the change affects pricing, otherwise the update is applied immediately.

Special Requirements
- Ticket pricing must support multiple tiers (e.g., general admission, VIP).
- Artists should be notified promptly (e.g., push/email) of an approval or rejection decision.

Frequency of Occurrence
Moderate – expected whenever an Artist plans a new event.

Repeat the section above for each additional team member, with their own 3 use cases in the same fully-dressed format.

7.2 Garrett DeYong – Use Cases

7.2.1 UC: Make a Post – Artist

Use case: Make a post
Actor: Artist
Preconditions:

1. The artist is logged in.

2. The artist is allowed to post.

Postconditions;

1. A new post belonging to the user is saved in the database.
2. This post can be viewed and commented on according to provided settings.

Main flow:

1. The artist selects the option to create a post.
2. The system prompts the artist to provide post content (text, files) and post permissions (whether it is public, whether it can be commented on).
3. The artist provides this information.
4. The system stores the post.
5. The system adds the post to the artist's account.

Extensions:

4a. No post content provided

4a1. The system determines no post content was provided.

4a2. The system prompts the sender to provide post content.

Special requirements:

- For public posts, the system should update the artist's profile content quickly (within 5 seconds, ideally).

7.2.2 UC: Message Another User (Beginning a New Conversation)

Use case: Message another user (beginning a new conversation)
Actor: User (artist or fan)
Preconditions:

1. The sender is logged in.
2. The sender is allowed to send messages.
3. The sender has accessed the recipient's page.
4. The recipient is accepting messages (both in general and from the sender specifically).
5. The sender and recipient do not currently have an ongoing conversation.

Postconditions:

1. A new conversation thread is established.

Main flow:

1. The sender selects the option on the recipients profile to begin a message.

2. The system prompts the sender to type the message.
3. The sender inputs the message.
4. The system checks and validates the message.
5. The system creates a new conversation thread with the provided message.
6. The system enables access to this thread to the sender and the recipient.

Extensions:

4a. Message empty

4a1. The system determines no message was provided.
4a2. The system prompts the sender to type the message.

4b. Message not appropriate/allowable

4b1. The system determines the message is not appropriate in some way.
4b2. The system warns the sender of this and prompts for message modifications.

Special requirements:

- The recipient should be notified that they have received a new message.

7.2.3 UC: Monitor Comments – Artist

Use case: Monitor comments
Actor: Artist
Preconditions:

1. The artist is logged in.
2. The artist has accessed the comment section of one of their posts.

Postconditions:

1. A given comment has been hidden from users.
2. Admins have been informed, if the artist so chooses.

Main flow:

1. The artist selects the manage option on a comment.
2. The system prompts the artist to select the desired action (delete/report/block user).
3. The artist provides this information.
4. The system hides the comment from users.
5. If reporting was selected, the system files a report.
6. If blocking the user was selected, the system makes a request for this.

Extensions: None
Special requirements:

- The system should not delete the comment from the database; it should be stored, but not visible to non-admin users.
- The comment should be hidden quickly (within 2 seconds, ideally).
- The user should be blocked (if applicable) quickly (within 2 seconds, ideally).
- The report (if applicable) should be sent within a few minutes.

7.3 Nicholas Nolen – Use Cases

7.3.1 UC: Update Event

Primary Actor
Artist

Scope Artist Home Page

Level User goal

Stakeholders & Interests
- **Fan:** As a fan I want to be able to view artist's most up to date event information.
- **Artist:** As an artist I want to provide up to date information about my events.

Preconditions
- An artist has logged into the system.
- An artist has already created an event.

Postconditions (Success Guarantee)
- An event has been updated on the artist's profile.

Main Success Scenario
1. The artist selects the edit event option in their profile.
2. The system prompts the artist to write over the event information (e.g. event type, description, date, time, relevant images/flyers).
3. The artist updates their event's information.
4. The system validates the input.
5. The system prompts the artist to review their updated event details.
6. The artist acknowledges they reviewed their details and submits the event.
7. The event shows up on the artist's profile to everyone who views it.

5a. Invalid Input

Extensions
- The system detects invalid input.
- The system displays an error message and requests the artist correct the issue.

6a. Artist fails to acknowledge
- The system displays an error message stating the artist must acknowledge they have reviewed the details of their submission before submitting.
- The artist acknowledges their review.
- The artist submits their event.

Frequency of Occurrence
Mdderate – It is expected for artists to make mistakes uploading event information.

7.3.2 UC: Unban Fan

Primary Actor
Admin

Scope Administrative service

Level System goal

Stakeholders & Interests
- **Admin:** As an admin I want to do my job and process appeals
- **Fan:** As a fan I want to be appealed to access the site.

Preconditions
- An Admin has logged into the system.
- A ban appeal has been made by a fan.

Postconditions (Success Guarantee)
- A different admin than the banning admin has reviewed the appeal.
- Fans are either unbanned or the ban is upheld.

Main Success Scenario
1. The admin selects the option to review open appeals from bans performed by admin other than themselves.
2. The admin selects an appeal.
3. The admin reviews fan submitted reasoning for the appeal along with related report material.
4. The admin reviews the automated AI recommendation for how to proceed based on ToS.
5. The admin selects an option to proceed with unbanning the user.
6. The system removes the user's ban.
7. The system grants the user the ability to log into the website.

Extensions
1a. No appeals
- The system shows no open appeals.
- The admin closes the reports tab.

5a. Appeal is invalid
- The admin selects an option to mark the appeal as invalid.
- The system logs the appeal as invalid and removes it from the open appeals list.

Frequency of Occurrence
High – There will likely always be fans trying to appeal bans.

7.3.3 UC: Recieve Reccomendation

Primary Actor
Fan

Scope Fan Home Page

Level User goal

Stakeholders & Interests
- **Fan:** As a fan I want to be able to find artists similar to those I like.
- **Artist:** As an artist I want to get my art out there to fans.

Preconditions
- A fan has logged into the system.
- A fan has previously interacted with artists on the platform.

Postconditions (Success Guarantee)
- A fan receives recommendations for artists they might like.

Main Success Scenario
1. A fan selects the option to display artists they might like.
2. The system retrieves the artists a fan has previously interacted with (e.g. viewed profiles, liking posts, commenting on posts, etc.).
3. The system constructs a numerical score for each of the types and genres of content the user interacts with.
4. The system uses an AI process to find artists with similar content to the user's most interacted with types.
5. The system displays a list of artists to the user.

Extensions 1a. New Fan
- The fan has logged into the system for the first time and does not have interactions.
- The system does not show the option to display artists they might like.

Frequency of Occurrence
High – It is likely fans will want frequent reccomendations.

7.4 Gan-Orgil Gantumur – Use Cases

7.4.1 UC: Suspending Artist Account – Admin

Use case: Suspending Artist Account
Actor: Administrator
Preconditions:

1. The Administrator is logged into the system.
2. A user account has been reported or flagged for potential terms of service (ToS) violations.

Postconditions:

1. The user's account is successfully suspended.
2. The system revokes the user's access and permissions.

Main flow:

1. The Administrator requests to view reported user accounts.
2. The system provides the pending reports and associated user activity.
3. The Administrator determines the user's actions violate the ToS.

4. The Administrator submits a request to suspend the user.
5. The system updates the account status to suspended and revokes the user's permissions.

Extensions:

3a. Report does not violate ToS

3a1. The Administrator determines the user's actions do not violate the ToS.

3a2. The Administrator dismisses the report, and the system maintains the account's active status.

7.4.2 UC: Comment on Artwork – Fan

Use case: Comment on Artwork
Actor: Fan
Preconditions:

1. The Fan is logged in with "user" role permissions.
2. An Artist has published media that allows comments.

Postconditions:

1. The Fan's comment is saved and published on the Artist's post.

Main flow:

1. The Fan requests to add a comment to an Artist's media.
2. The system prompts the Fan for the comment content.
3. The Fan provides the comment content.
4. The system validates the comment against business rules.
5. The system saves and publishes the comment to the Artist's page.

Extensions:

4a. Content flagged by system business rules

4a1. The system determines the content violates business rules (e.g., via basic word filters or optional AI-based content moderation, if enabled).

4a2. The system rejects the comment and notifies the Fan.

5a. Inappropriate comment reported post-publication

5a1. An Artist monitors their comments and identifies a published comment as inappropriate.

5a2. The Artist submits a report regarding the comment for Administrator review.

7.4.3 UC: Approve Artist Verification Application – Admin

Use case: Approve Artist Verification Application
Actor: Administrator
Preconditions:

1. The Administrator is logged in and authorized to manage accounts.
2. A user has submitted an application to upgrade to the Artist role.

Postconditions:

1. The user's application is resolved (approved or rejected).
2. If approved, the user is granted the Artist role and associated permissions.

Main flow:

1. The Administrator requests pending Artist verification applications.
2. The system provides the submitted application materials.
3. The Administrator evaluates the application to verify the applicant's identity.
4. The Administrator approves the application.
5. The system assigns the user the "Artist" role and updates their permissions.

Extensions:

3a. Application fails verification

3a1. The Administrator determines the application is illegitimate or incomplete.
3a2. The Administrator rejects the application.
3a3. The system maintains the user's current role and notifies them of the rejection.

7.5 William Waweru – Use Cases

7.5.1 UC: Follow Artist – Fan

Use case: Follow Artist
Actor: Fan
Preconditions:

1. The Fan is logged into the system.
2. The Artist has an active account that is visible to the Fan.
3. The Fan is not currently following the Artist.

Postconditions:

1. The Fan is recorded as a follower of the Artist.
2. The Artist is added to the Fan's list of followed Artists.

Main flow:

1. The Fan requests to follow an Artist.
2. The system verifies that the Artist's account is active.
3. The system verifies that the Fan is not already following the Artist.
4. The system creates a following relationship between the Fan and the Artist.
5. The system updates the Artist's follower information.
6. The system confirms that the Fan is now following the Artist.

Extensions:

2a. Artist account is unavailable.

- 2a1. The system determines that the Artist's account is suspended, banned, or otherwise unavailable.
- 2a2. The system does not create the following relationship.
- 2a3. The system informs the Fan that the Artist cannot currently be followed.

3a. Fan already follows the Artist.

- 3a1. The system detects an existing following relationship.
- 3a2. The system does not create a duplicate relationship.
- 3a3. The existing following relationship remains unchanged.

7.5.2 UC: Manage Fan Profile – Fan

Use case: Manage Fan Profile
Actor: Fan
Preconditions:

1. The Fan is logged into the system.
2. The Fan has an existing account and profile.

Postconditions:

1. Valid profile changes are stored by the system.
2. The updated information is associated with the Fan's profile.
3. Profile information is displayed according to the Fan's privacy settings.

Main flow:

1. The Fan requests to modify their profile.
2. The system provides the Fan's existing profile information.
3. The Fan provides updated profile information, such as a biography, artistic interests, supported Artists, or profile image.

4. The Fan specifies applicable privacy preferences.
5. The Fan submits the updated profile information.
6. The system validates the submitted information according to profile and content rules.
7. The system stores the valid changes.
8. The system applies the Fan's privacy preferences to the updated information.
9. The system confirms that the profile has been updated.

Extensions:

6a. Invalid profile information.

6a1. The system identifies invalid profile information.
6a2. The system rejects the affected changes and informs the Fan which information must be corrected.
6a3. The Fan provides corrected information.
6a4. The process resumes at Step 5.

6b. Profile content violates platform rules.

6b1. The system determines that the submitted content violates platform content rules.
6b2. The system rejects the prohibited content.
6b3. The system informs the Fan that the profile cannot be updated with that content.

7a. Profile update cannot be stored.

7a1. The system preserves the previously stored profile information.
7a2. The system informs the Fan that the update was unsuccessful.

7.5.3 UC: Report Inappropriate Content – Fan

Use case: Report Inappropriate Content
Actor: Fan
Preconditions:

1. The Fan is logged into the system.
2. The content or account being reported exists and is accessible to the Fan.

Postconditions:

1. A new report is stored with a pending status.
2. The report contains the reporting Fan, reported content or account, reason, and relevant information.
3. The report is available for Administrator review.

Main flow:

1. The Fan identifies content or user behavior that they believe violates the platform's rules.
2. The Fan requests to report the content or account.
3. The system identifies the content or account associated with the report.
4. The system requests a report reason and any additional relevant information.
5. The Fan provides a reason and submits the report.
6. The system validates that the report contains the required information.
7. The system creates the report and records the relevant information.
8. The system assigns the report a pending status and makes it available for Administrator review.
9. The system confirms to the Fan that the report has been submitted.

Extensions:

3a. Reported content is no longer available.

- 3a1. The system determines that the reported content is no longer publicly available.
- 3a2. If sufficient information about the original content remains available, the system allows the report to continue.
- 3a3. Otherwise, the system informs the Fan that the report cannot be completed.

6a. Report does not contain required information.

- 6a1. The system determines that required report information is missing.
- 6a2. The system does not submit the report.
- 6a3. The system requests the missing information from the Fan.

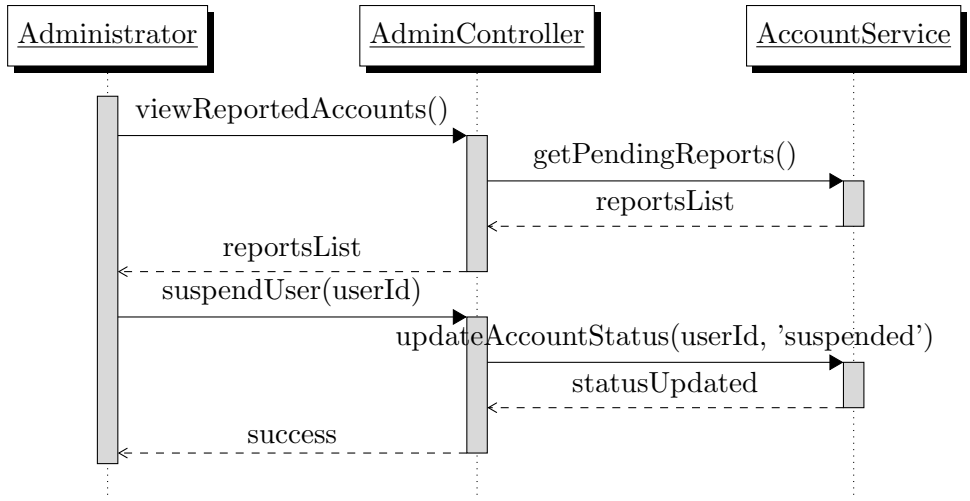
7a. Duplicate report exists.

- 7a1. The system identifies an existing pending report from the same Fan for the same content or account.
- 7a2. The system prevents creation of a duplicate report.
- 7a3. The system informs the Fan that they have already reported the content or account.

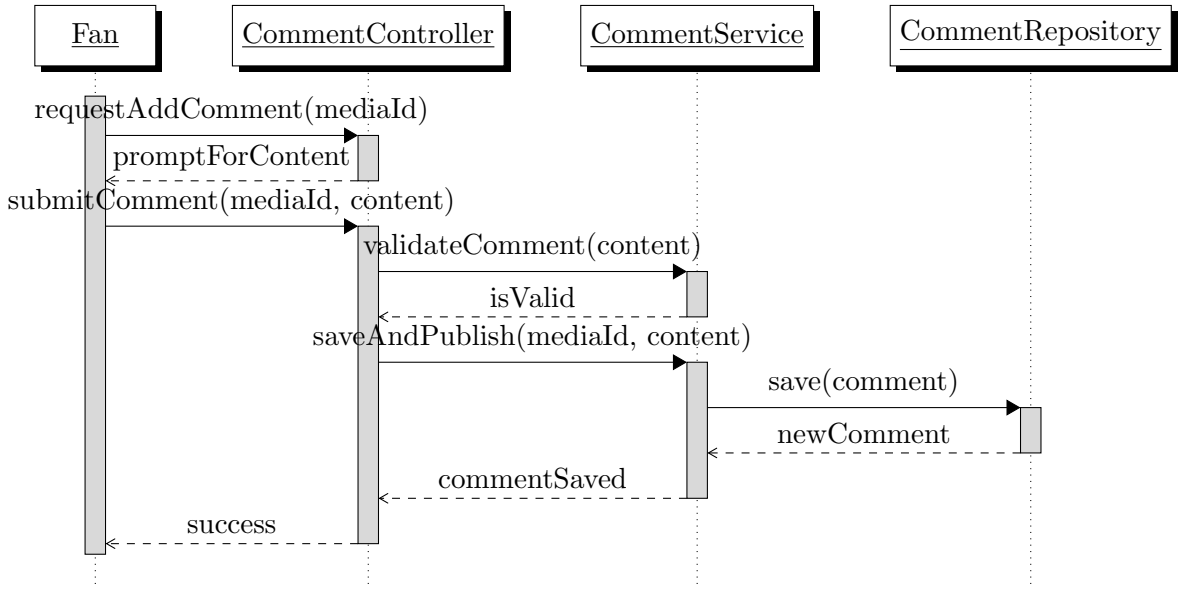
8. Sequence Diagrams

8.1 Garrett DeYong – Sequence Diagrams

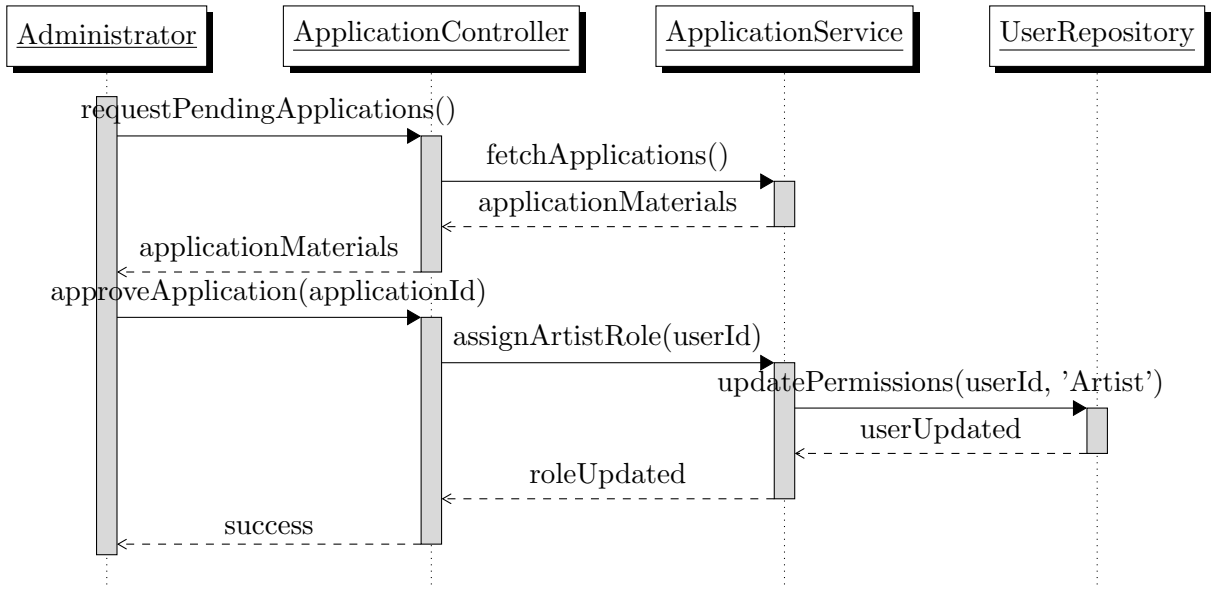
Administrator - Suspending Artist Account



Fan - Comment on Artwork

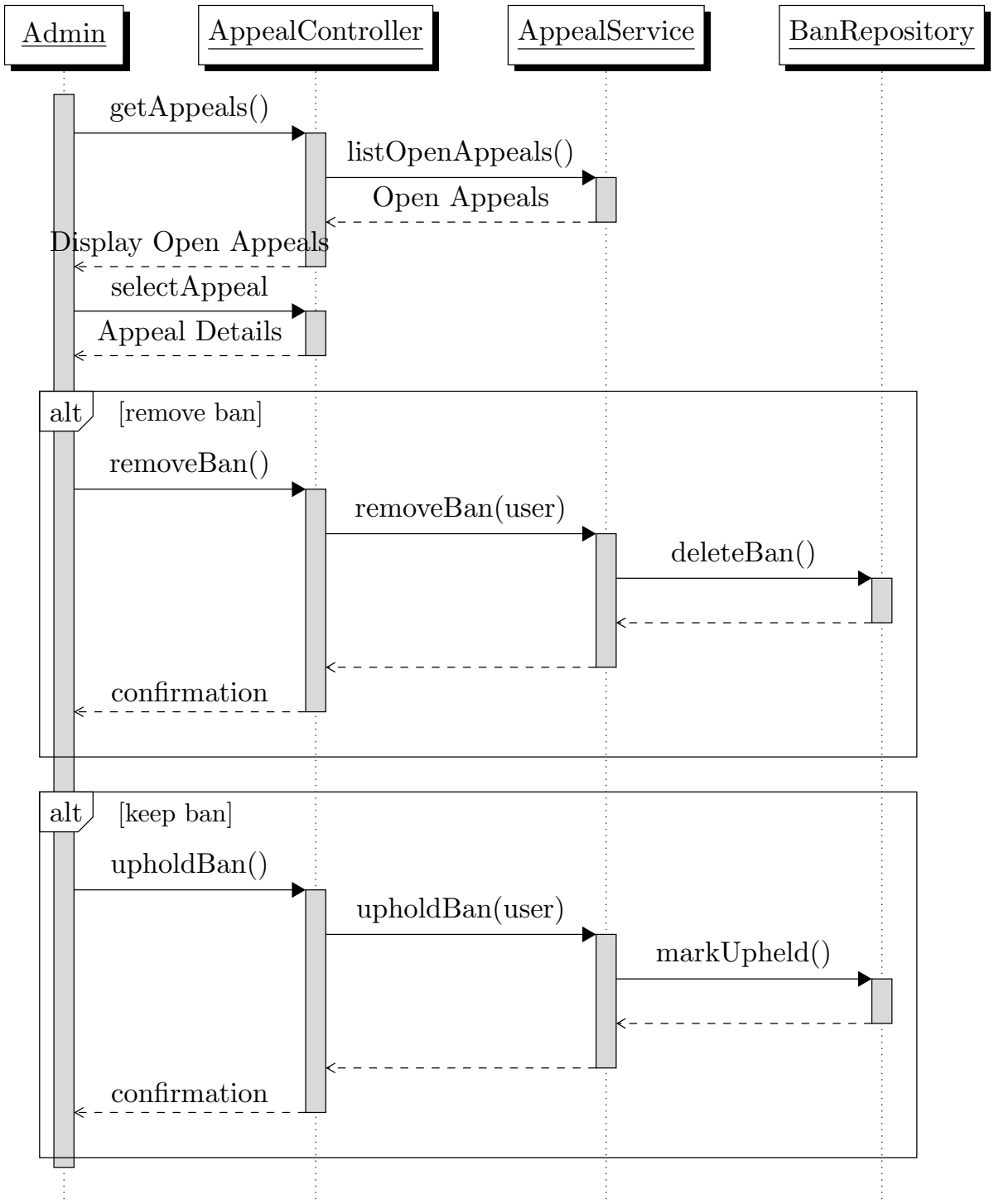


Administrator - Approve Artist Verification Application

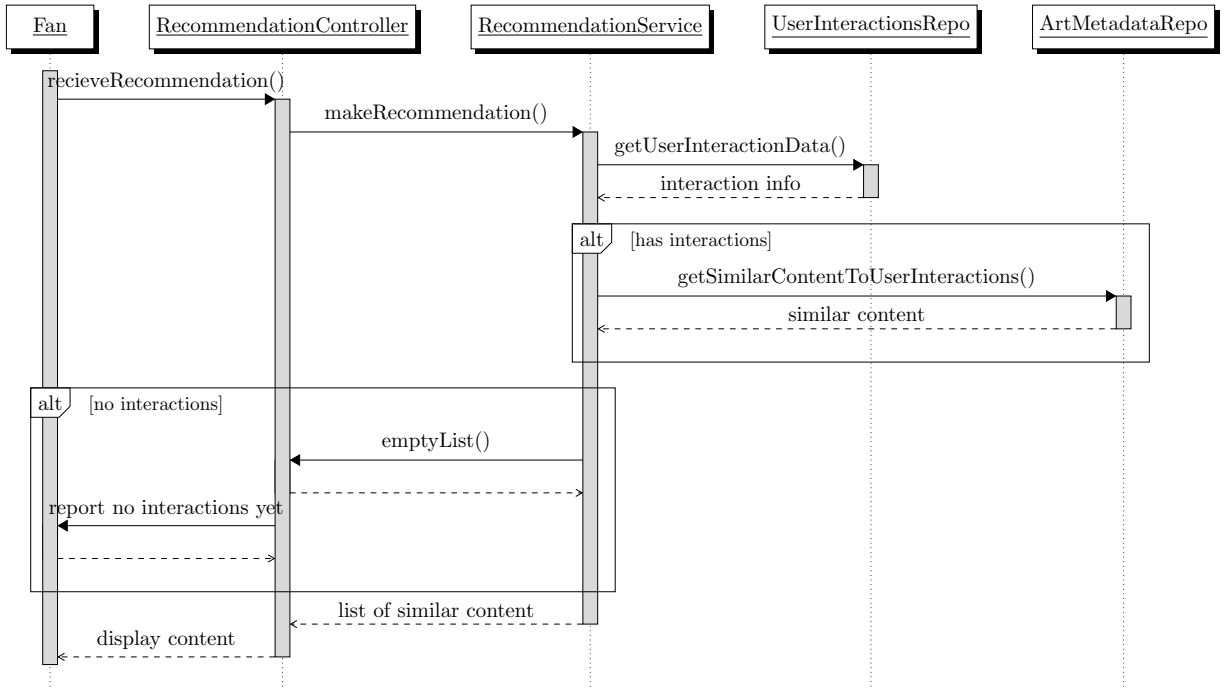


8.2 Nicholas Nolen – Sequence Diagrams

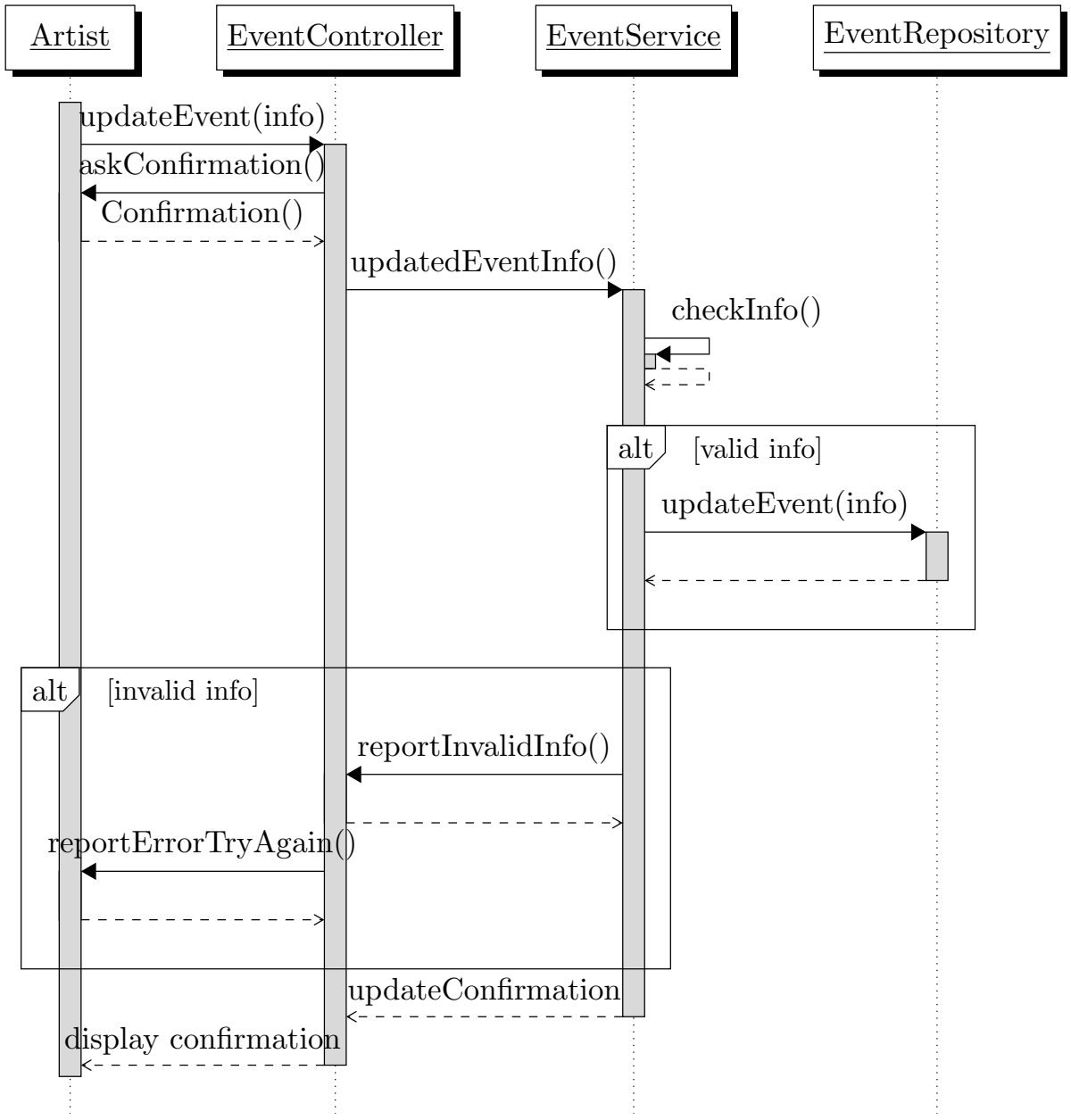
Admin - Unban Fan (Appeals)



Fan - Receive Recommendation

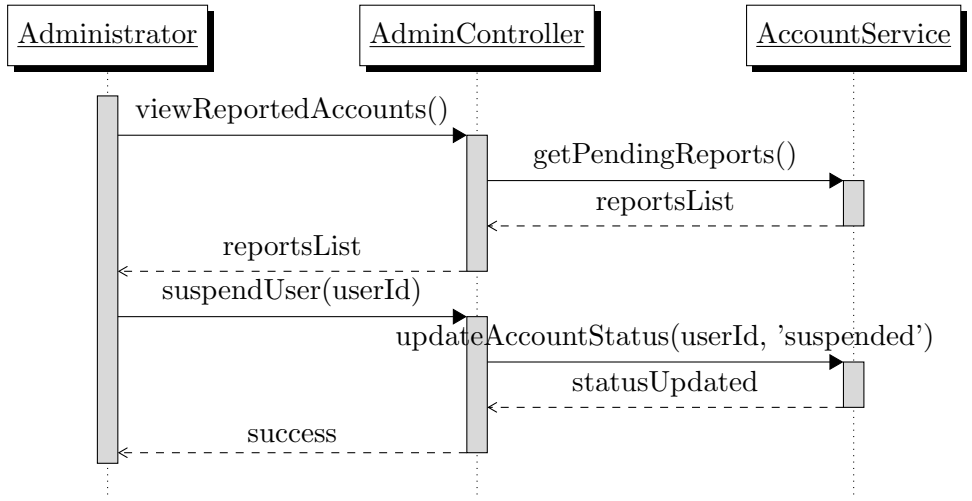


Artist - Update Event

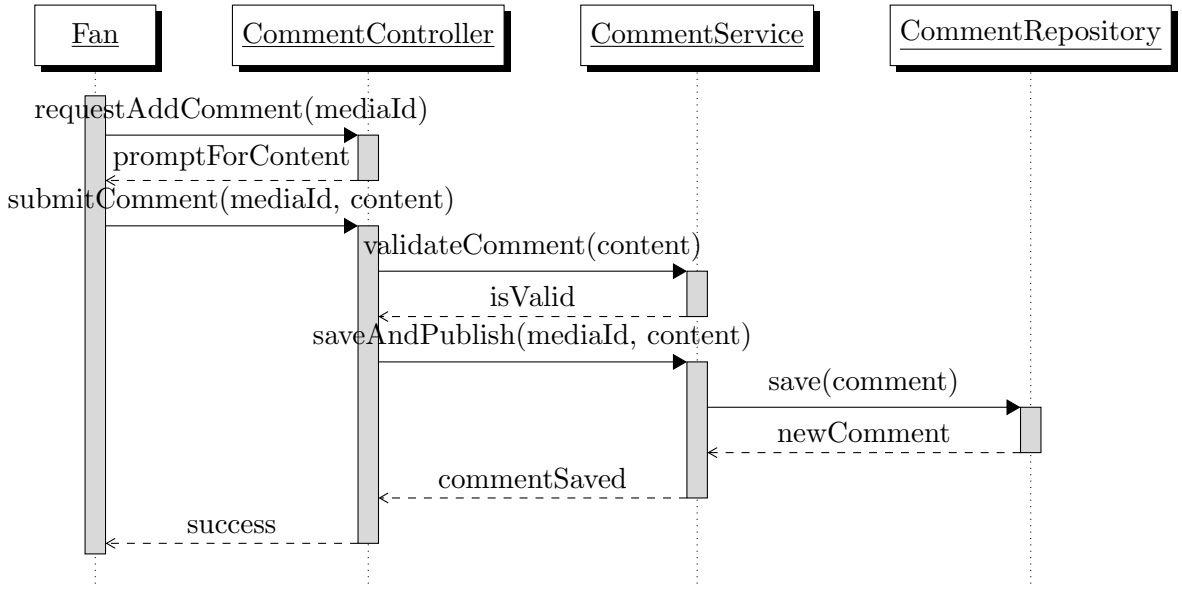


8.3 Gan-Orgil Gantumur – Sequence Diagrams

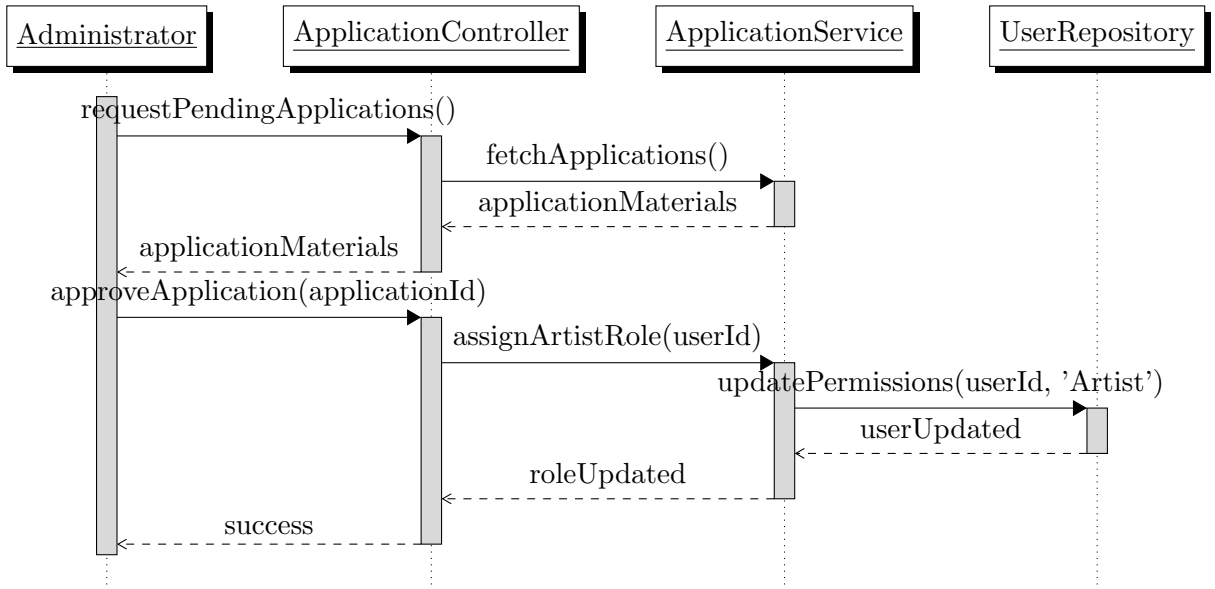
Administrator - Suspending Artist Account



Fan - Comment on Artwork

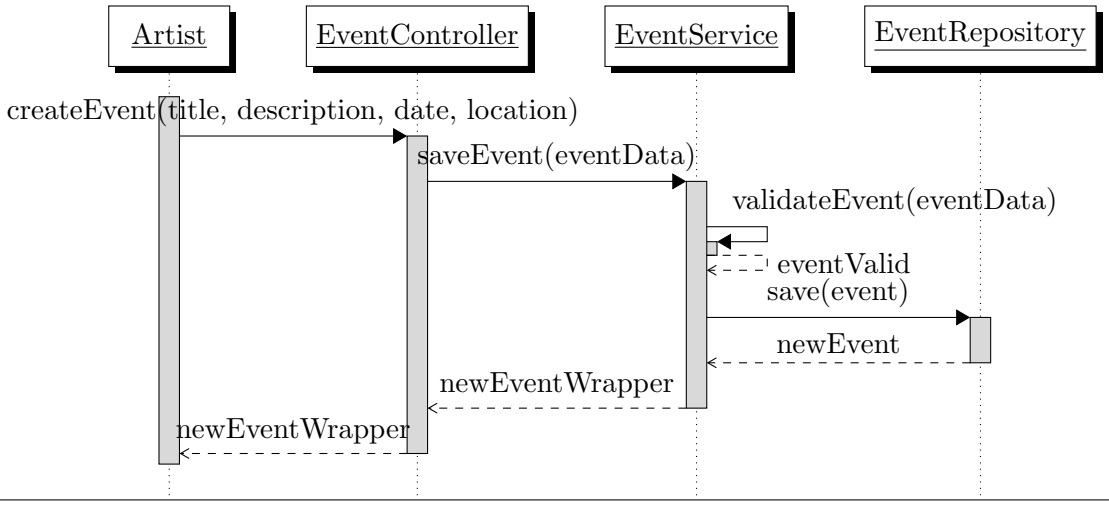


Administrator - Approve Artist Verification Application

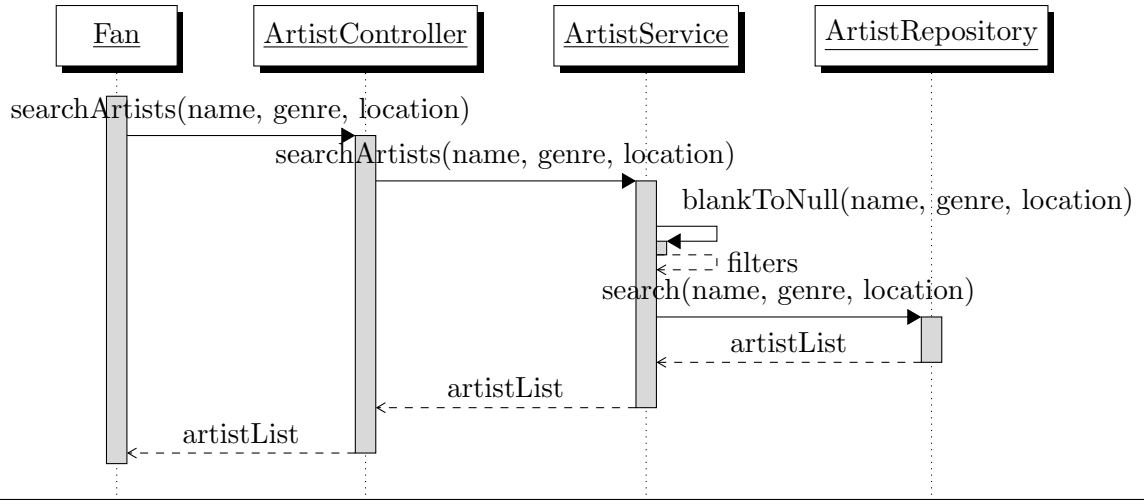


8.4 Alex Garcia – Sequence Diagrams

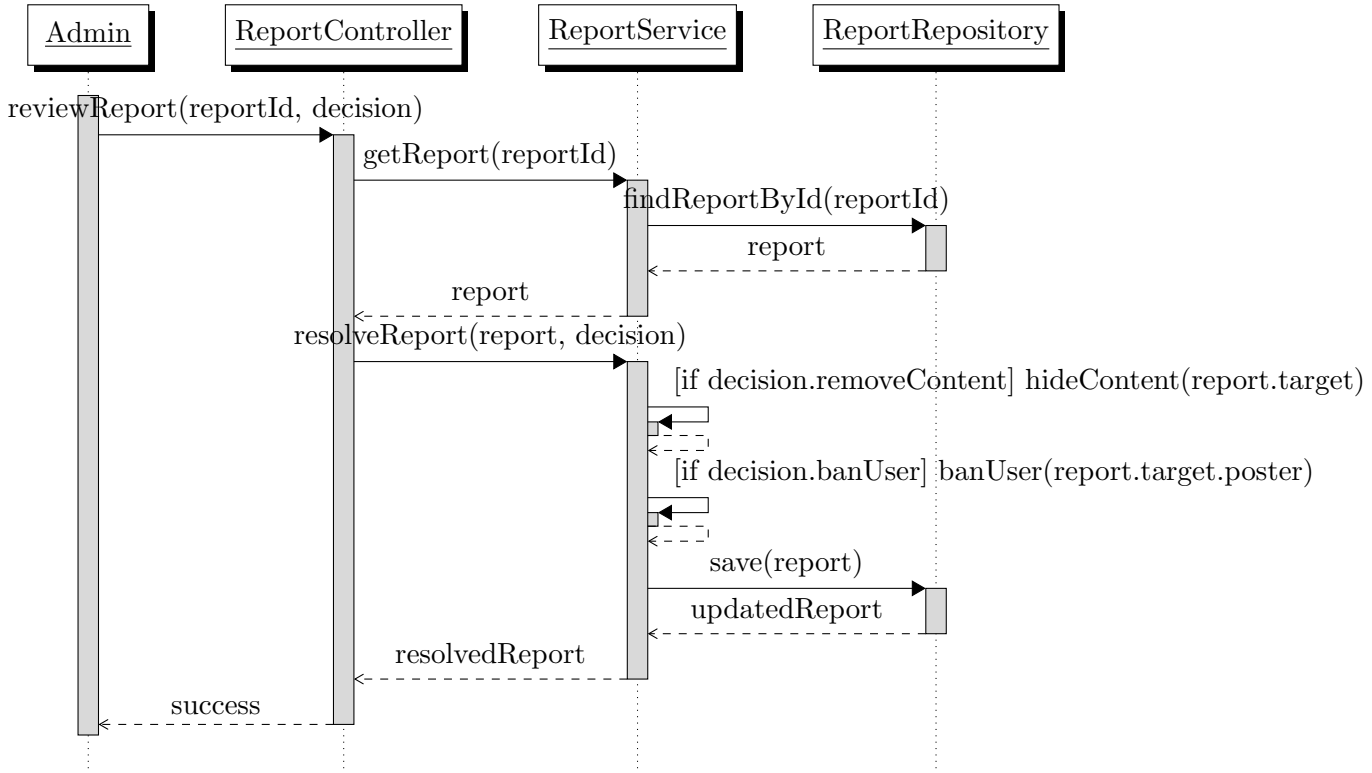
Artist - Create Event



Fan - Search for Artist

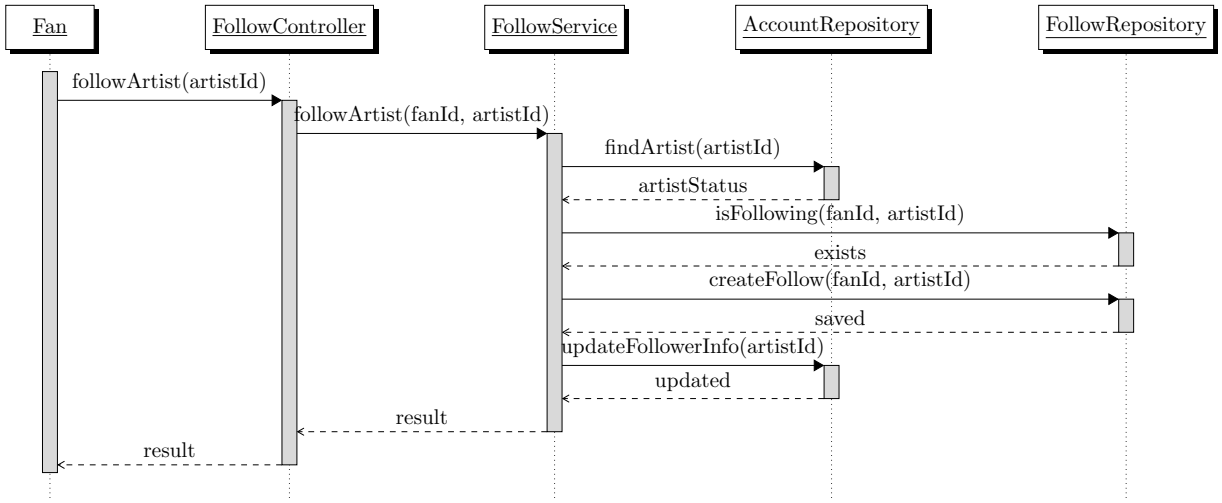


Admin - Review Report

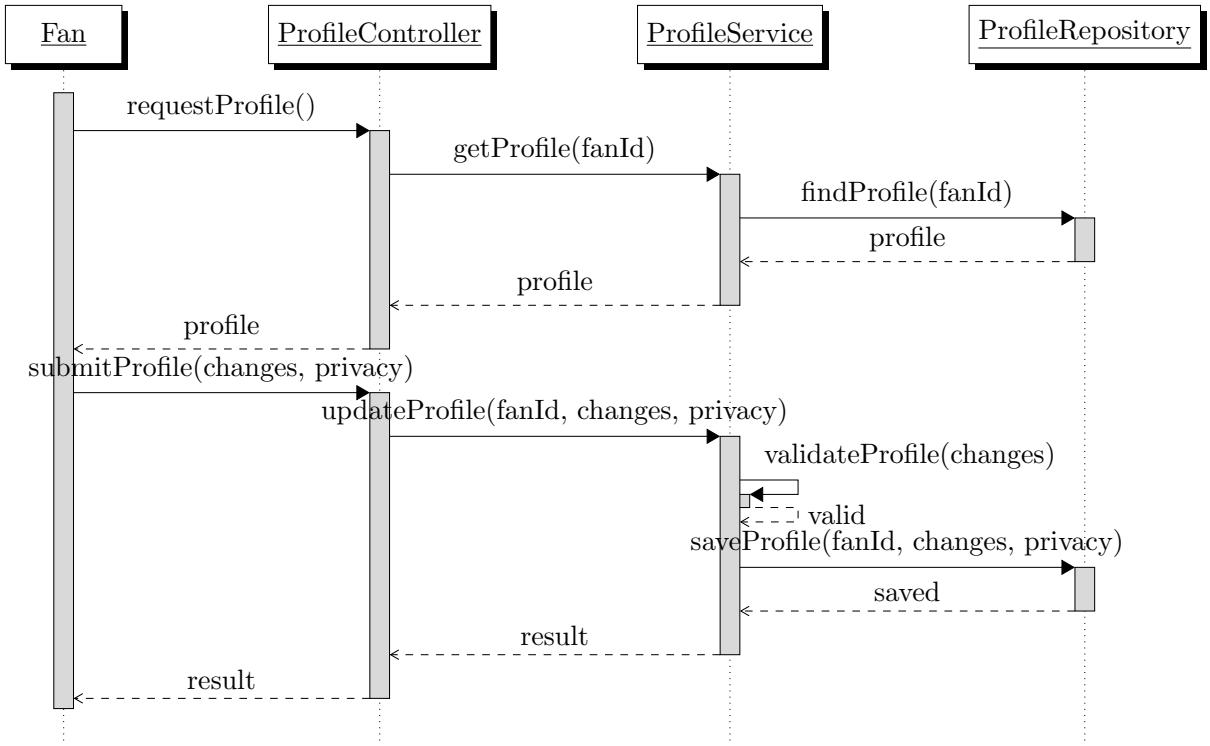


8.5 William Waweru – Sequence Diagrams

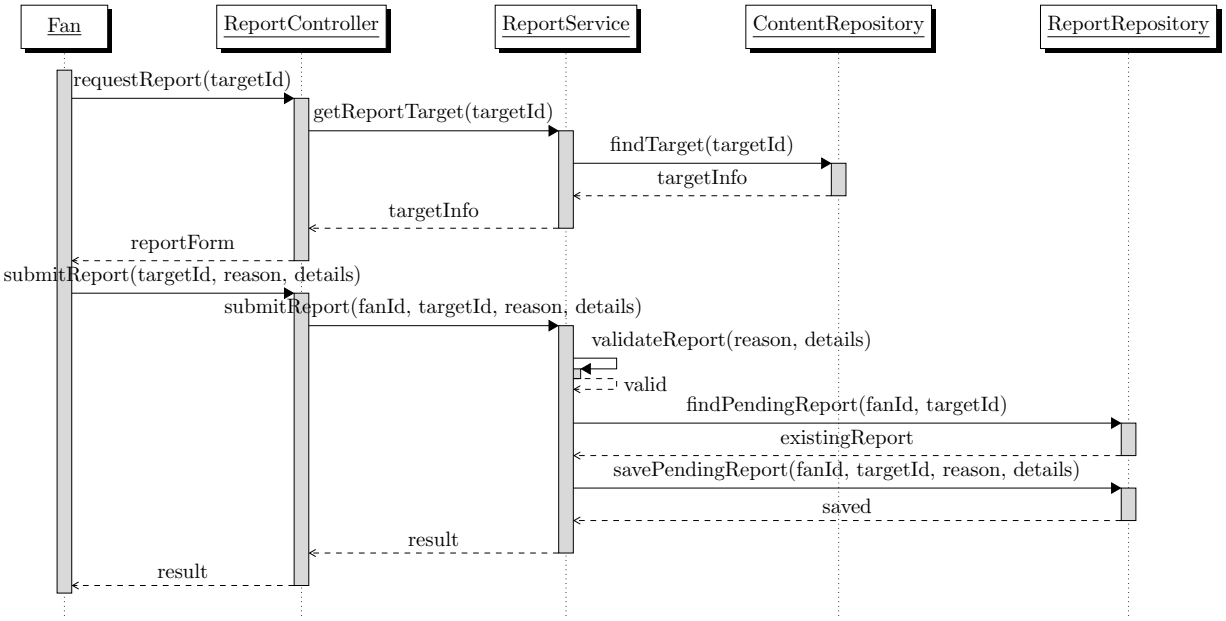
8.5.1 Follow Artist – Fan



8.5.2 Manage Fan Profile – Fan



8.5.3 Report Inappropriate Content – Fan



9. Use Case Diagram

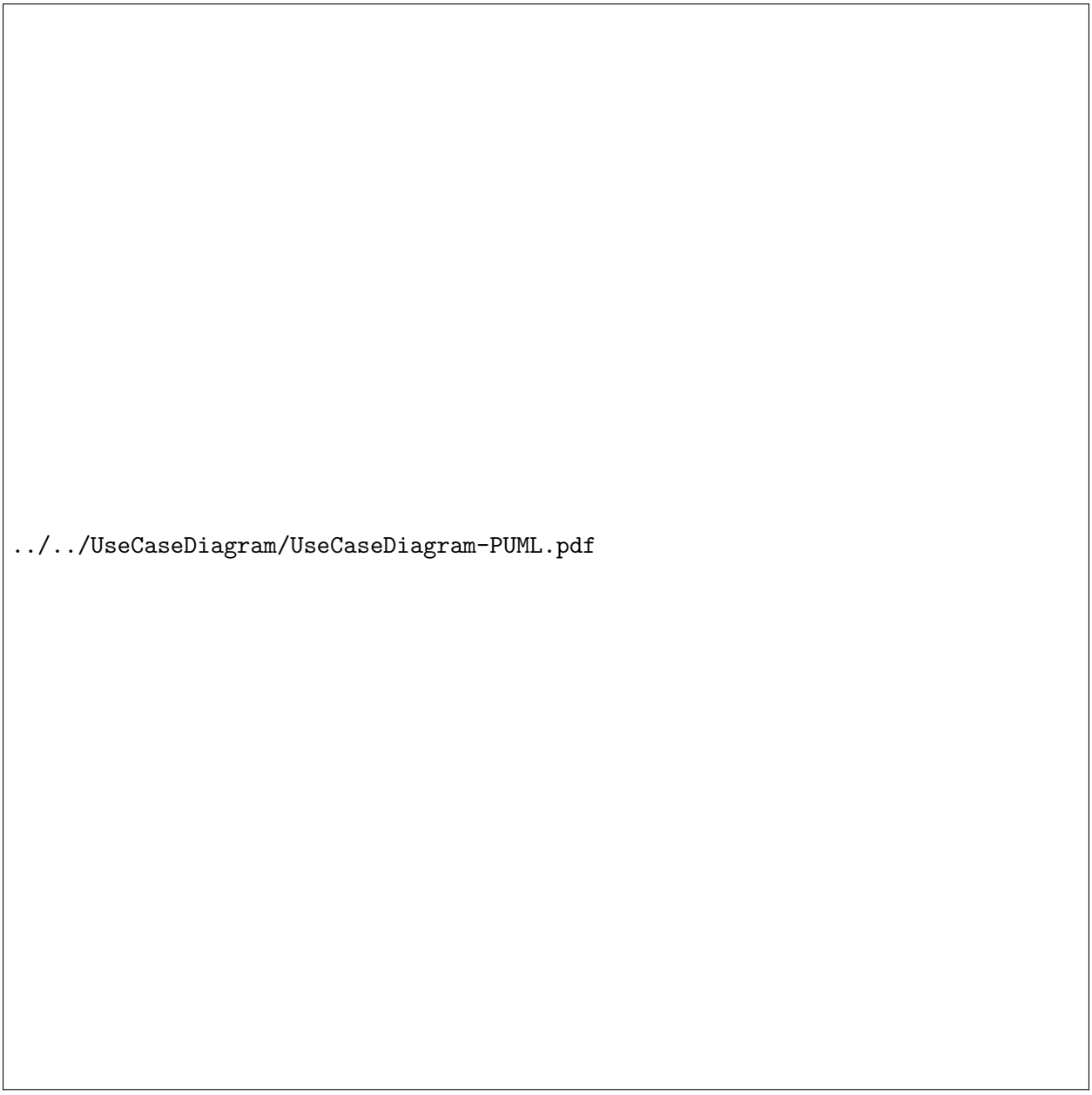
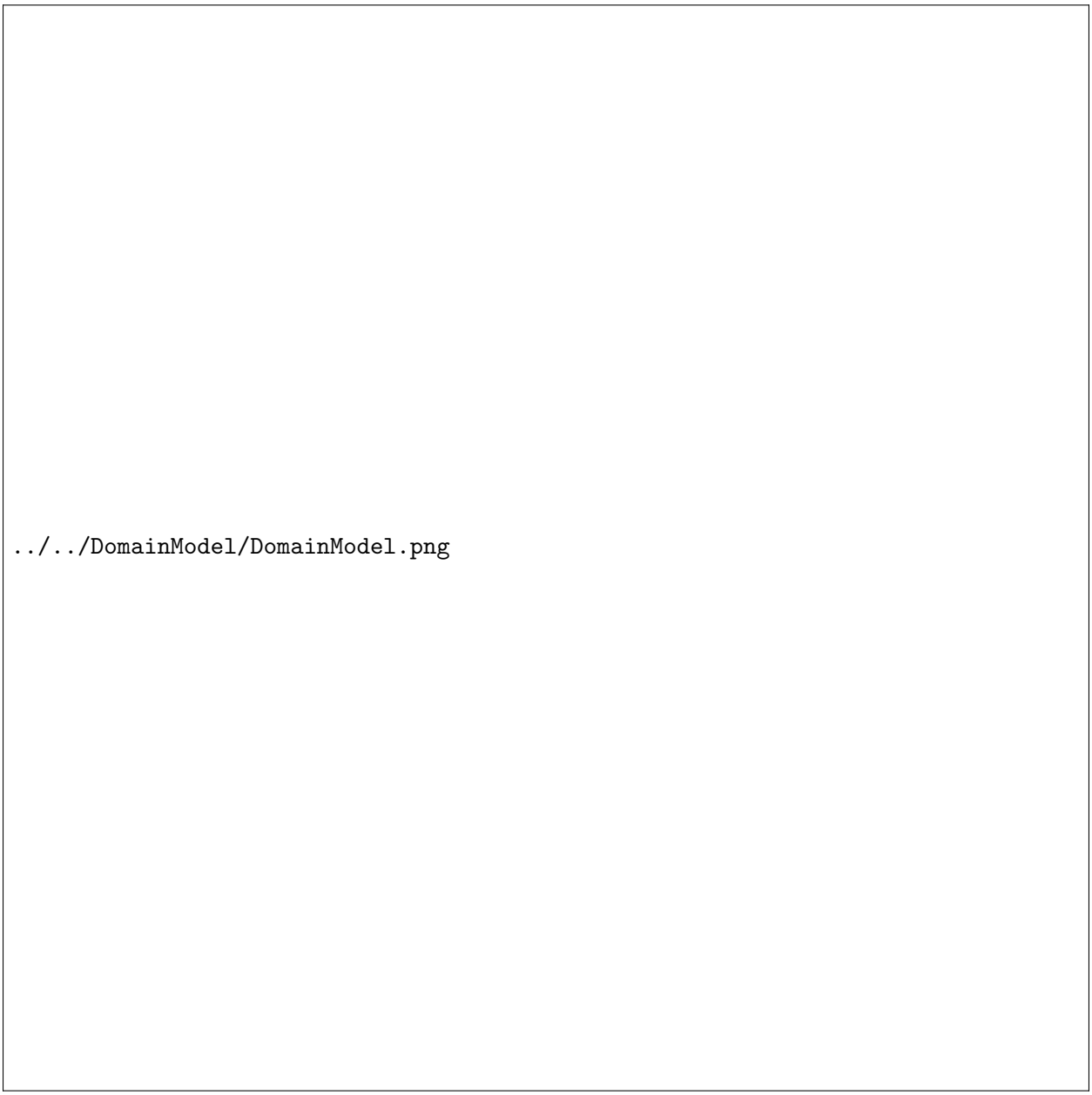


Figure 9.1: System Use Case Diagram

10. Domain Model



11. Operation Contracts

Operation: searchArtists(criteria: SearchCriteria): List⟨ArtistSummary⟩

Cross Reference

UC: Search for Artist – Fan

Preconditions

- The Fan is logged into the system.

Postconditions

- A collection of Artist instances matching the given criteria (name, genre, location) was retrieved.
- No system state was created, modified, or deleted (read-only operation).

Operation: resolveReport(reportId: ReportID, action: ActionType): void

Cross Reference

UC: Review Report – Admin

Preconditions

- The Admin is logged in with administrative privileges.
- A Report instance identified by `reportId` exists with status = "Pending."

Postconditions

- The Report instance's `status` attribute was changed to "Resolved."
- A new Resolution instance was created and associated with the Report, recording the acting Admin, the action taken, and a timestamp.
- If `action` = DeleteContent, the associated Message, Profile, or Event instance's `status` attribute was changed to "Removed."
- If `action` = Suspend or Ban, the reported User instance's `status` attribute was changed to "Suspended" or "Banned" respectively.
- A new Notification instance was created and associated with the reporting User.
- A new Notification instance was created and associated with the reported User, if `action` $\neq$ Dismiss.

Operation: submitEvent(eventData: EventForm): Event

Cross Reference

UC: Create Event – Artist

Preconditions
- The Artist is logged in and owns an account in good standing.

Postconditions
- A new Event instance was created with `title`, `dateTime`, `location`, `description`, `capacity`, and `proposedPrices` set from `eventData`.
- The new Event instance's `status` attribute was set to "Pending Approval."
- An association was formed between the new Event instance and the Artist who created it.
- The new Event instance was added to the Admin approval queue.

12. Test Cases

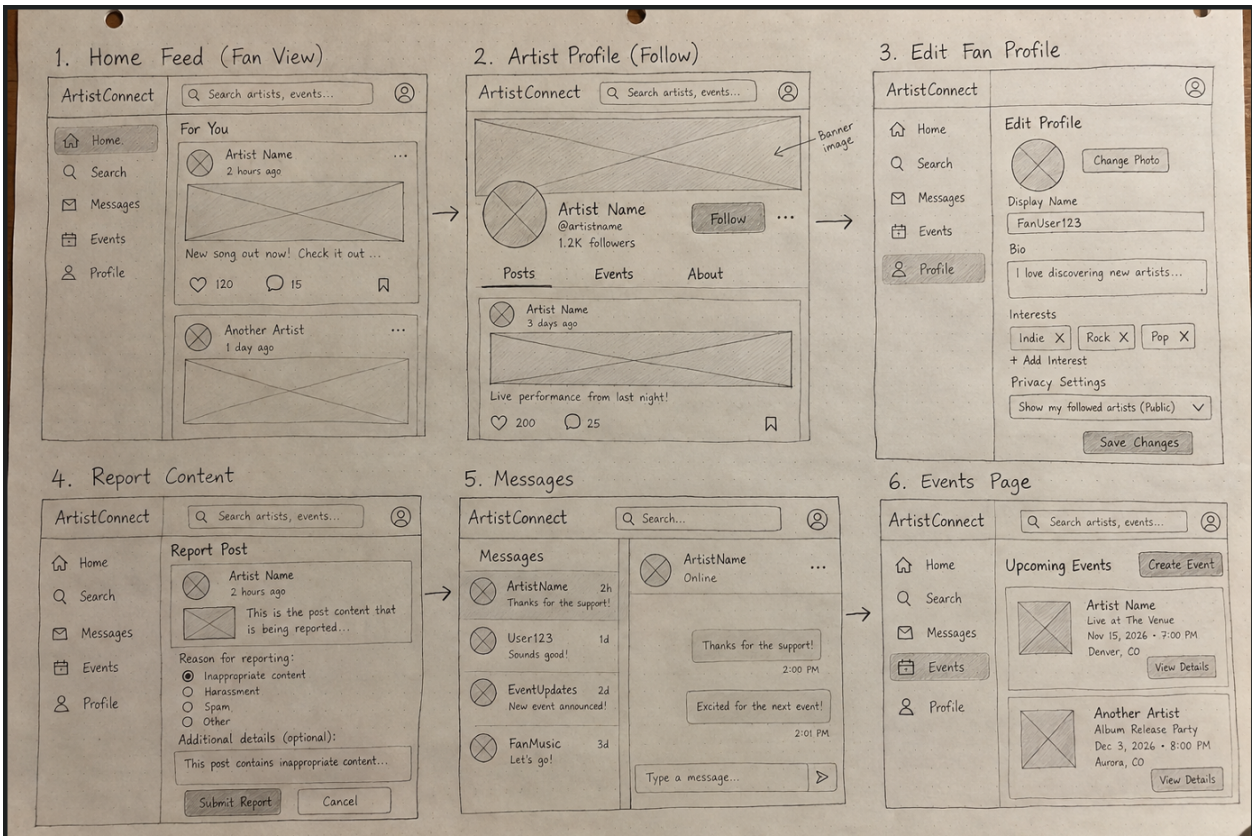
12.1 Test Cases for Iteration 1

No test cases were produced during iteration one.

12.2 Test Cases for Iteration 2

- Artist Search
- Unban Fan
- Edit Event

13. UI Mockups



13.1 Make a Post – Artist

Figure 13.1 shows the artist's post creation interface, including the story field, media upload control, visibility settings, comment preferences, and a live preview.

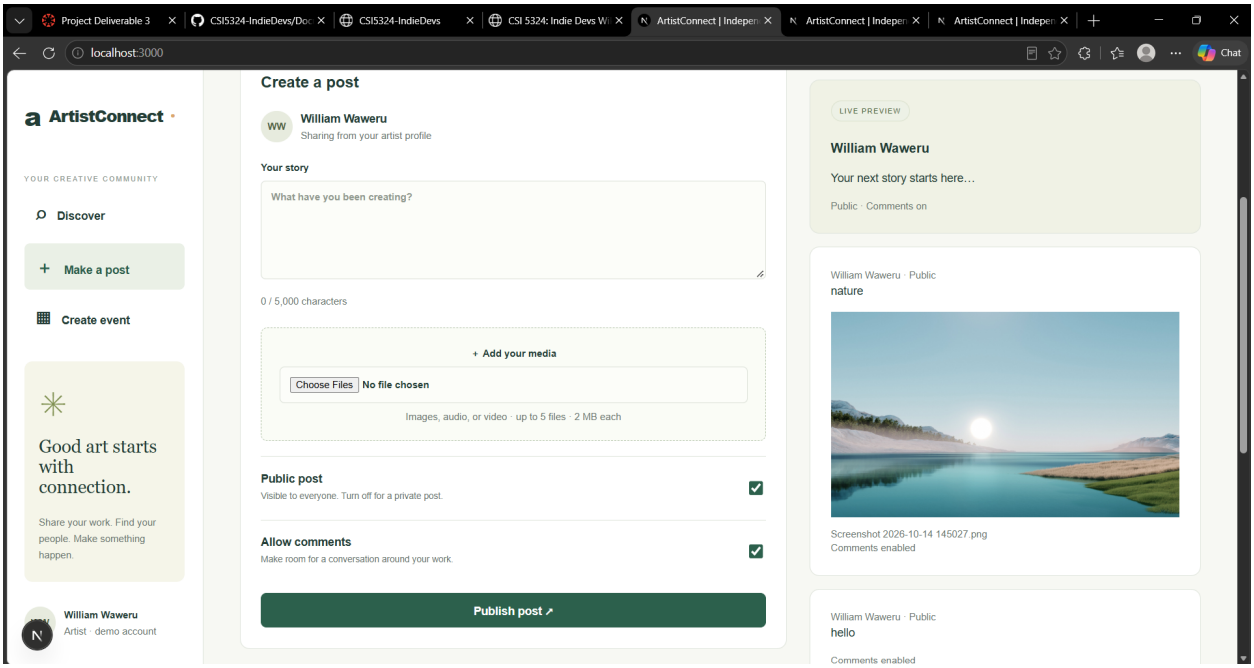


Figure 13.1: ArtistConnect UI preview for the Make a Post use case.

13.2 Search for Artist – Fan

Figure 13.2 shows the artist discovery interface, where a fan can search using artist name, genre or style, and location, then view matching artist profiles.

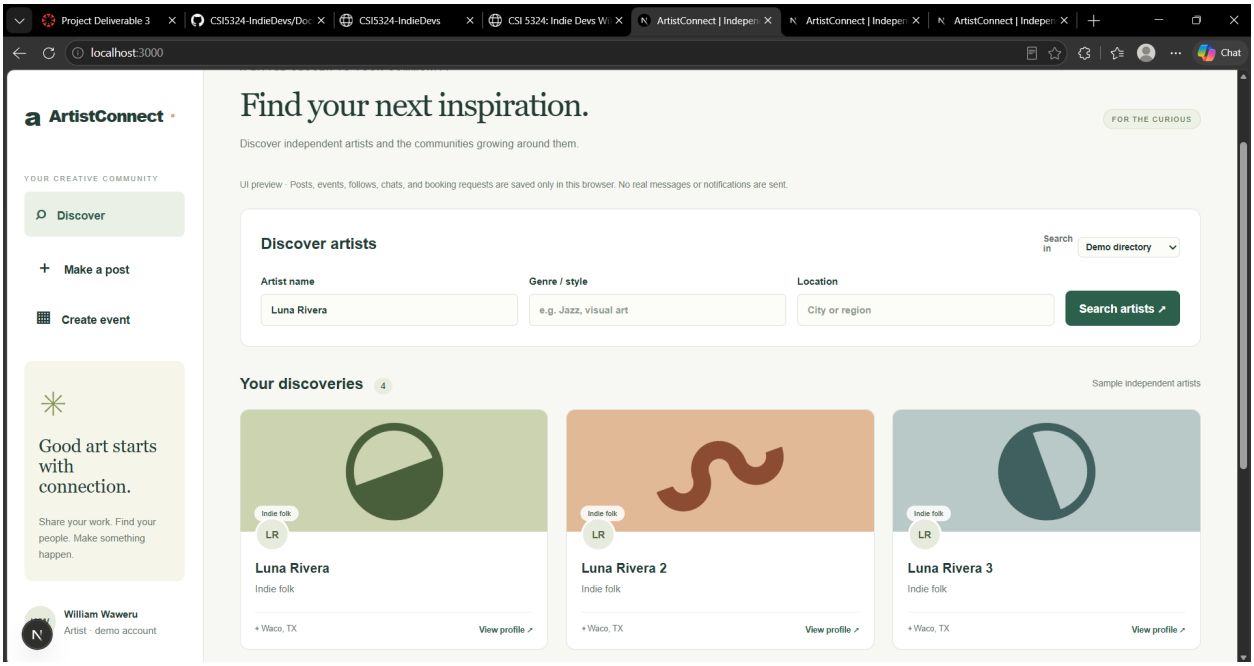


Figure 13.2: ArtistConnect UI preview for the Search for Artist use case.

13.3 Create Event – Artist

Figure 13.3 shows the event creation interface, including event details, venue, capacity, description, ticket tiers, and draft or review submission actions.

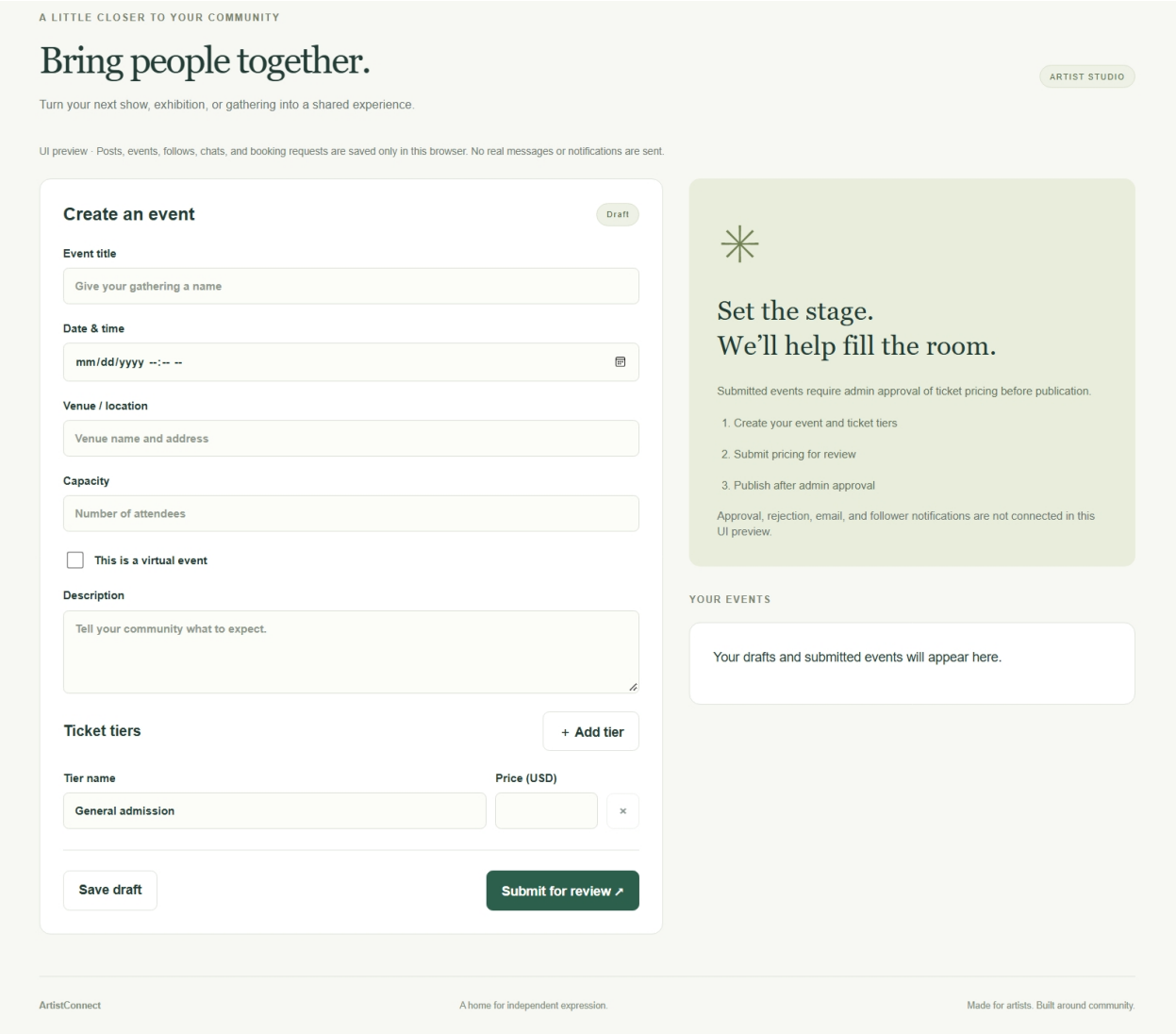


Figure 13.3: ArtistConnect UI preview for the Create Event use case.